Kenjaku: A Local-First Riichi Mahjong Replay-Analysis Toolkit

gongahkia

Draft built from repository source on 2026-07-07

Abstract

Riichi mahjong is a multi-player imperfect-information game whose research workflows often depend on replay logs, external engines, and platform-specific data constraints. Kenjaku is a local-first Python toolkit that narrows the current goal to reproducible replay analysis: parsing local Tenhou XML, reconstructing supervised decision examples, training small baseline models, exporting neutral decision snapshots, comparing prediction files, and documenting permission boundaries. This draft does not claim playing strength, production readiness, complete yaku validation, or complete Sanma coverage. Instead, it documents the current architecture and the evidence that is tracked in the repository: 360 unit tests with 15 skips, fixture-only launch media, a fixture benchmark dashboard, and runbooks for ignored local experiments. The main contribution is therefore an auditable workflow boundary for future mahjong-AI research rather than a final agent.

1 Introduction

Riichi mahjong replay analysis targets the reconstruction and evaluation of player decisions from game logs. This setting is useful for supervised policy learning, post-game review, benchmark reporting, and model comparison, but it is hard to make public and reproducible because real replay logs may carry platform, redistribution, and player-data restrictions. Kenjaku addresses this workflow problem by separating checked-in code and synthetic fixtures from ignored local replay exports, reports, and model artifacts.

The strongest published mahjong systems and public engines show that the game is a serious imperfect-information AI benchmark. Suphx demonstrated deep reinforcement learning for mahjong, including global reward prediction, oracle guiding, and run-time policy adaptation [1]. Mortal and akochan provide practical Japanese-mahjong engine baselines and review tooling [2, 3, 4, 5]. More recent projects such as MahjongLM and Mahjax explore language-model-style mahjong rollouts and GPU-accelerated JAX simulation [6, 7, 8, 9]. These systems motivate Kenjaku's long-term direction, but they also make a boundary problem visible: a repository that wants to compare against external engines must avoid copying incompatible code, importing weights without review, or presenting live-service automation as a default workflow.

Kenjaku's current contribution is a compliance-aware research harness. The toolkit implements a small but test-covered path from local XML to decision examples, snapshot JSONL, prediction comparison, browser demo artifacts, and sandbox experiments. The external-baseline path is deliberately process-based: real Mortal-compatible or akochan-compatible predictors are expected to run outside Kenjaku and communicate through neutral JSONL files. The data path is similarly conservative: raw Tenhou XML, downloaded databases, checkpoints, and generated reports stay under ignored directories unless release terms are reviewed.

The evidence in this draft is intentionally modest. The repository contains synthetic Tenhou fixtures, public-safe launch media generated from those fixtures, and local runbooks for larger

ignored experiments. It does not contain TODO-601 final evaluations, real engine-strength Mortal or akochan outputs, or a trained release checkpoint. All claims below are scoped to tracked artifacts and commands, and missing evidence is listed as future work rather than implied performance.

2 Related Work

Deep reinforcement learning for mahjong. Suphx is the most important published reference for high-strength mahjong AI in this report. The paper frames mahjong as a multi-player imperfect-information challenge and reports a deep-RL system with global reward prediction, oracle guiding, and run-time policy adaptation [1]. Kenjaku does not reproduce Suphx and does not claim comparable strength; it uses Suphx as the research context for why replay data, evaluation protocols, and platform boundaries matter.

Open-source riichi engines and review systems. Mortal describes itself as a free open-source Japanese-mahjong AI powered by deep reinforcement learning, with documentation and a GitHub implementation [2, 3]. akochan is another Japanese-mahjong AI implementation [4], and mjai-reviewer exposes a review workflow around mjai-compatible engines including Mortal and akochan [5]. Kenjaku's distinction is not engine strength; it is the neutral snapshot and subprocess boundary documented in docs/external-baselines.md.

Simulation and representation frameworks. Mahjax presents a fully vectorized JAX environment for large-scale riichi-mahjong rollout on GPUs [8, 9]. Mjx is another open-source mahjong framework for AI research [10]. MahjongLM explores mahjong language models and rollout datasets [6, 7]. Kenjaku is complementary: it is currently a Python replay analysis and baseline-reporting toolkit, not a high-throughput simulator or language-model training stack.

Replay data and platform constraints. Tenhou publishes manuals and service pages, and its reporting page includes a category for unauthorized AI players [11, 12]. Local Tenhou workflows can also use external downloader tooling such as houou-logs [13], but Kenjaku treats downloaded databases and exported XML as local-only artifacts. This policy is tracked in docs/data-policy.md and docs/local-tenhou-eval.md.

3 Method

3.1 Workflow Overview

Kenjaku represents the current replay-analysis workflow as a sequence of narrow, inspectable modules. Given local Tenhou XML, it parses game events, reconstructs draw/discard/call/riichi decision examples, trains small baselines, exports neutral decision snapshots, and compares prediction JSONL files keyed by stable row identifiers. The CLI also writes public-safe replay summaries, static browser-demo assets, benchmark dashboards, and sandbox reports.

The central design choice is that every module has a data-boundary role. Parser and reconstruction code can be tested on synthetic fixtures. Larger local datasets remain ignored. External engines communicate through subprocesses and JSONL rather than through imported code. Public artifacts summarize aggregate metrics or synthetic fixtures rather than raw private replays.

3.2 Data Governance

Kenjaku separates source-code reproducibility from data redistribution. The repository permits small synthetic fixtures under `data/fixtures/`; raw Tenhou exports, downloader databases, processed shards, reports, and model checkpoints belong under ignored paths such as `data/raw/`, `runs/`, and `models/`. The replay-intake commands require a manifest with platform, URI, intended uses, and permission status before any row enters an analysis queue.

This module is necessary because replay logs are not just model inputs. They can contain platform references, player-derived data, and redistribution constraints. The advantage of the manifest gate is that it makes permission scope part of the artifact pipeline, not a side note in a README.

3.3 Decision Reconstruction And Snapshots

Kenjaku reconstructs supervised examples for discard, call/pass, and riichi/pass decisions from local Tenhou XML. It then exports decision snapshots containing stable `row_id` values, legal actions, observed actions, reconstruction fields, and a minimal mjai-style event prefix. The snapshot format is intentionally neutral: a Kenjaku model, a deterministic stub, or an external producer can write prediction JSONL without sharing code.

This design supports fairer comparison work because every producer reads the same rows and returns the same action schema. It also avoids future-outcome leakage by excluding terminal labels by default and requiring an explicit outcome flag when such labels are needed for analysis.

3.4 Baselines, Reports, And Dashboard Artifacts

The current repository contains dependency-free frequency and linear baselines for discard, call, riichi, and deal-in experiments, plus PyTorch model scaffolding for discard MLP and transformer policy experiments. Benchmark reports preserve train/eval counts, split metadata, metric definitions, feature summaries, calibration diagnostics where implemented, and source-command metadata. The public dashboard converts report JSON into a static HTML page with metric definitions and explicit live-ladder exclusions.

The baseline layer is not presented as a strong agent. Its purpose is to validate data plumbing, metric computation, and artifact structure before larger ignored local runs and external-engine comparisons.

3.5 Training Commands And Ablation Hooks

Kenjaku's training commands are intentionally small and report-oriented. The dependency-free discard benchmark can compare frequency, raw-count linear, shanten-aware linear, risk-context linear, defense-context linear, and defense-context-v1 variants on one deterministic split. Call and riichi benchmarks expose threshold calibration and weighted variants. Neural commands train a small discard MLP or a transformer discard policy head when PyTorch is available.

These commands provide ablation hooks rather than final ablation evidence. A meaningful ablation requires a permitted dataset slice and enough held-out decisions; the fixture dashboard only proves that the model-family rows and ablation fields render. TODO-601 must replace this smoke evidence with final supervised, self-play, Sanma, and interpretability tables before strength claims are valid.

Table 1: Fixture-only launch-media evidence generated from `data/fixtures/tenhou`. These numbers are smoke evidence for artifact wiring, not model-strength evidence.

Artifact	Verified content
Replay-analysis clip	8 valid decision snapshots, 0 malformed rows, 4 discard decisions, 1 call decision, 3 riichi decisions, 8 mjai-event prefixes present.
Prediction comparison	8 echo-actual predictions, 0 missing predictions, 0 malformed predictions, 1.0000 overall accuracy by construction.
Benchmark dashboard image	Fixture discard benchmark with 4 examples, 3 train examples, 1 eval example, and four fast baseline rows.
Browser-demo clip	Rendered local browser demo page generated by `kenjaku browser-demo --no-serve`.

3.6 External-Engine Boundary

The external-baseline path uses a subprocess boundary. Kenjaku sets `KENJAKU_SNAPSHOTS` and `KENJAKU_PREDICTIONS`; an external producer writes prediction rows; Kenjaku validates and compares those rows. This design is meant for Mortal-compatible, akochan-compatible, or other producers whose code, license, weights, or runtime artifacts should remain outside the repository.

This module is necessary because importing AGPL or otherwise constrained engine code would change Kenjaku's licensing and redistribution surface. A separate process keeps the comparison protocol usable while preserving legal and architectural separation.

4 Experiments And Evidence

4.1 Tracked Verification

The strongest tracked verification is the unit-test suite and the fixture-only launch-media run. On 2026-07-07, the repository passed `python3.13 -m ruff check .`, `PYTHONPATH=src python3.13 -m compileall -q src tests`, and `PYTHONPATH=src python3.13 -m unittest discover -s tests`. The unittest run executed 348 tests with 15 skipped tests. The launch-media run generated docs/media/replay-analysis.mp4, docs/media/browser-demo.mp4, and docs/media/benchmark-summary.png from synthetic fixtures.

4.2 Fixture Benchmark

The checked launch-media benchmark uses `data/fixtures/tenhou` and runs the fast discard models: frequency, linear, risk-context linear, and defense-context linear. All four rows report 1.0000 eval accuracy on a one-example eval split, while train accuracy varies from 0.3333 to 1.0000. This result is useful only as a dashboard and metric-smoke artifact. It is too small for model selection, claims about playing strength, or claims about defense feature value.

4.3 Ablation Scope

The fixture report includes ablation fields for risk-context lift over the linear model and defense-context lift over the risk-context model. In the fixture run, both eval lifts are +0.0000 because the eval split has one example and every fast model predicts it correctly. This confirms that ablation fields are present in the report schema, but it does not establish causal value for any feature family.

4.4 Ignored Local Runs

The repository records larger local workflows in runbooks rather than checking in restricted data. docs/local-tenhou-eval.md gives commands for Tenhou XML exports, discard/call/riichi benchmarks, decision snapshots, disagreement reports, and dashboard generation. docs/external-baselines.md records a prior ignored external baseline smoke over 86,003 shared decision snapshots, but explicitly labels Mortal-compatible and akochan-compatible entries as deterministic protocol baselines rather than real engine-strength outputs. These notes support reproducibility planning, not public benchmark claims.

5 Compliance And Data Policy

Kenjaku's public release policy has four rules. First, raw replay data and processed datasets that can reconstruct restricted logs stay out of git. Second, platform-specific replay intake must pass through manifest review before analysis. Third, public demo artifacts must avoid raw private replay URLs and private player/account data. Fourth, live-service automation remains out of scope unless a platform grants explicit permission for the exact integration.

This policy is not just legal caution; it is part of the experimental method. Without it, model metrics could be hard to reproduce publicly, impossible to audit, or unsafe to redistribute. The current repository therefore prefers synthetic fixtures, source commands, ignored local artifacts, and aggregate summaries over checked-in raw logs.

6 Limitations

Kenjaku is not a trained production mahjong agent. The repository does not bundle a strong policy, does not contain final TODO-601 evaluation tables, does not include real Mortal or akochan engine-strength comparisons, and does not claim to match or exceed supervised baselines. The current dashboard image is generated from four fixture examples, so it is a smoke artifact only.

Rules coverage is also incomplete. The sandbox contains substantial plumbing for riichi, furiten, kan timing, several yaku metadata paths, abortive draws, narrow Sanma support, and an exact fu/han payment scorer with expanded yaku legality, but it is not a complete yaku validator and not a complete Tenhou Sanma implementation. The README and status command intentionally preserve those negative claims and list unsupported yaku explicitly.

7 Claim-Evidence Map

Table 2: Major claims in this draft and the tracked evidence that supports or limits them.

Claim	Evidence	Status
Kenjaku is local-first and avoids raw replay redistribution.	docs/data-policy.md, docs/local-tenhou-eval.md, and data/README.md.	Supported
The CLI can generate fixture launch media and a benchmark dashboard.	docs/launch-media.md and tracked media under docs/media/.	Supported
The fixture snapshot path emits 8 valid rows with 0 malformed rows.	docs/launch-media.md; reproduced with `export-decision-snapshots` and `decision-snapshot-summary`.	Supported
The repository has broad unit-test coverage for current primitives and commands.	2026-07-07 local run: 360 tests, 15 skipped; command recorded in README launch evidence.	Supported
Kenjaku has exact fu/han payment scoring.	`src/kenjaku/core/scoring.py`, `tests/test_core_scoring.py`, and status capability `full_scoring_engine`.	Supported
Kenjaku has expanded yaku legality.	`src/kenjaku/core/yaku.py`, `tests/test_core_yaku.py`, and status unsupported-yaku list.	Supported
Kenjaku has a neutral external-baseline boundary.	docs/external-baselines.md and CLI commands `run-external-prediction-producer` and `external-baseline-report`.	Supported
Kenjaku is a strong mahjong-playing agent.	No bundled strong policy or final evaluation artifact exists.	Not claimed
Kenjaku matches Mortal or akochan.	Real engine-strength producer outputs are not tracked.	Not claimed
Kenjaku has complete yaku validation and complete Sanma rules.	Unsupported yaku remain listed in status, and open GitHub issue TODO-401 remains.	Not claimed

8 Self-Review

Contribution. The draft contributes a documented, compliance-aware replay-analysis workflow, not a new mahjong-strength result. This is a meaningful systems contribution for the repository, but it would not be sufficient as a standalone research paper without TODO-601 final evaluations.

Writing clarity. The method is reproducible from listed CLI commands and tracked docs. The main terminology is stable: "local-first", "decision snapshots", "external producer", and "public-safe artifacts".

Experimental strength. The tracked experiments are weak by design. Fixture evidence validates wiring, media generation, and report structure only. The paper explicitly avoids strength claims.

Evaluation completeness. Final supervised, self-play, Sanma, and interpretability evaluations are missing. Issue TODO-601 must provide reproducible tables before this draft can become a final technical report.

Method design soundness. The conservative data and subprocess boundaries add friction, but they reduce licensing and redistribution risk. The main unresolved design question is how to run real external baselines without importing constrained code or weights.

9 Conclusion

Kenjaku currently provides a reproducible local workflow for riichi mahjong replay analysis rather than a complete mahjong agent. Its core idea is to keep parsing, reconstruction, snapshots, baseline reports, dashboards, and public-safe artifacts auditable while keeping raw replay data and external engines outside the repository. The tracked evidence supports this workflow boundary, but not playing strength. The next step is TODO-601: freeze permitted dataset slices, run final supervised and sandbox evaluations, replace smoke baselines with real external producer outputs where legally usable, and update this report with reproducible tables.

References

[1] Junjie Li, Sotetsu Koyamada, Qiwei Ye, Guoqing Liu, Chao Wang, Ruihan Yang, Li Zhao, Tao Qin, Tie-Yan Liu, and Hsiao-Wuen Hon. "Suphx: Mastering Mahjong with Deep Reinforcement Learning." arXiv:2003.13590, 2020. https://arxiv.org/abs/2003.13590.

[2] Mortal documentation. https://mortal.ekyu.moe/.

[3] Equim-chan. Mortal GitHub repository. https://github.com/Equim-chan/Mortal.

[4] critter-mj. akochan GitHub repository. https://github.com/critter-mj/akochan.

[5] Equim-chan. mjai-reviewer GitHub repository. https://github.com/Equim-chan/mjai-reviewer.

[6] Mitsuhiro Taniguchi. MahjongLM GitHub repository. https://github.com/MitsuhiroTaniguchi/MahjongLM.

[7] mitsutani. MahjongLM collection on Hugging Face. https://huggingface.co/collections/mitsutani/mahjonglm.

[8] Soichiro Nishimori et al. "Mahjax: A GPU-Accelerated Mahjong Simulator for Reinforcement Learning in JAX." arXiv, 2026. https://arxiv.org/html/2605.20577v1.

[9] nissymori. Mahjax GitHub repository. https://github.com/nissymori/mahjax.

[10] "Mjx: A framework for Mahjong AI research." IEEE Conference on Games, 2022. https://dl.acm.org/doi/10.1109/CoG51982.2022.9893712.

[11] Tenhou manual. https://tenhou.net/man/.

[12] Tenhou report page. https://tenhou.net/stat/vt/0/.

[13] Apricot-S. houou-logs GitHub repository. https://github.com/Apricot-S/houou-logs.